UNIVERSIDADE FEDERAL DE LAVRAS

PROGRAMA DE PÓS-GRADUAÇÃO EM ENGENHARIA DE SISTEMAS E AUTOMAÇÃO

COMPUTAÇÃO EVOLUCIONÁRIA

Implementação do Algoritmo Genético

Discentes:
Matheus Higino
Marcos Vinicius Cardoso Reis

Professor:
Bruno Henrique Groenner Barbosa

Lavras – MG
Abril de 2026

Sumário

1 Introdução **2**

2 Metodologia **2**
2.1 Representação Cromossomial 2
2.2 Problemas de Otimização 2
2.2.1 Cross-in-Tray 2
2.2.2 Drop-Wave 3
2.3 Operadores Genéticos 3
2.4 Configurações Avaliadas 3

3 Resultados **4**
3.1 Melhor Configuração Encontrada 4
3.2 Boxplot Comparativo das Melhores Configurações 5
3.3 Impacto da Probabilidade de Cruzamento 6
3.4 Impacto da Probabilidade de Mutação 6
3.5 Impacto do Método de Seleção 7
3.6 Impacto do Tamanho da População 7
3.7 Impacto do Número de Gerações 8
3.8 Impacto do Elitismo 8
3.9 Gráfico de Convergência 9

4 Análise dos Resultados **9**

5 Conclusão **10**

A Apêndice – Código-Fonte da Implementação **11**
A.1 Arquivo principal: `main.py` 11
A.2 Execução dos experimentos: `experiments/run_experiments.py` 12
A.3 Algoritmo Genético: `src/algoritmo_genetico.py` 14
A.4 População inicial: `src/populacao.py` 15
A.5 Representação cromossomial: `src/representacao.py` 15
A.6 Avaliação e funções objetivo: `src/fitness.py` 16
A.7 Seleção: `src/selecao.py` 17
A.8 Cruzamento: `src/crossover.py` 18
A.9 Mutação: `src/mutacao.py` 18
A.10 Estatísticas: `utils/estatisticas.py` 19
A.11 Geração de gráficos: `utils/graficos.py` 19
A.12 Análise final: `experiments/analise_final.py` 20
A.13 Boxplots por parâmetro: `experiments/boxplots_parametros.py` 23

1 Introdução

Os Algoritmos Genéticos (AGs) são técnicas de otimização inspiradas no processo de evolução natural e pertencem ao grupo dos algoritmos evolutivos. Seu funcionamento baseia-se em operadores como seleção, recombinação e mutação, simulando o processo evolutivo biológico com o objetivo de encontrar soluções ótimas ou próximas do ótimo global [1].

Essa abordagem foi amplamente consolidada na literatura como uma das principais estratégias para resolução de problemas complexos de busca e otimização, especialmente em cenários com múltiplos ótimos locais e espaços de busca extensos [2]. Sua principal vantagem está na capacidade de explorar diferentes regiões do espaço de soluções simultaneamente, reduzindo a dependência de condições iniciais específicas.

Neste trabalho foi implementado um Algoritmo Genético simples para a resolução de dois problemas de otimização com duas variáveis, conforme solicitado na disciplina de Computação Evolucionária. A implementação contemplou representação cromossomial binária, seleção por roleta viciada e torneio, cruzamento com um ponto de corte por variável, mutação bit a bit e elitismo opcional.

Além da implementação do algoritmo, foram realizados experimentos com diferentes combinações de parâmetros, incluindo probabilidade de cruzamento, probabilidade de mutação, método de seleção, tamanho da população, número de gerações, uso de elitismo e função objetivo. Para cada configuração, foram executadas 30 repetições, permitindo uma análise estatística mais robusta dos resultados obtidos.

2 Metodologia

2.1 Representação Cromossomial

Cada indivíduo foi representado por um cromossomo binário de 32 bits, dividido em duas partes:

- 16 bits para representar a variável x;
- 16 bits para representar a variável y.

Após a codificação binária, os valores foram convertidos para números reais no intervalo $[-10, 10]$, possibilitando a avaliação das funções objetivo.

2.2 Problemas de Otimização

Foram escolhidas duas funções clássicas da literatura de otimização contínua, ambas com duas variáveis e características multimodais.

2.2.1 Cross-in-Tray

$$f(x, y) = -0.0001 \left(\left| \sin(x) \sin(y) \exp \left(\left| 100 - \frac{\sqrt{x^2 + y^2}}{\pi} \right| \right) \right| + 1 \right)^{0.1} \tag{1}$$

A função Cross-in-Tray é conhecida por apresentar grande quantidade de ótimos locais, o que a torna particularmente desafiadora para algoritmos de busca.

2.2.2 Drop-Wave

$$f(x, y) = -\frac{1 + \cos(12\sqrt{x^2 + y^2})}{0.5(x^2 + y^2) + 2} \tag{2}$$

A função Drop-Wave também é multimodal, porém tende a apresentar comportamento mais regular do que a Cross-in-Tray.

Como o Algoritmo Genético foi estruturado como um problema de maximização de fitness, foi necessário adaptar as funções objetivo de forma adequada para esse contexto.

2.3 Operadores Genéticos

Foram implementados os seguintes operadores:

- **Seleção por roleta viciada**: baseada na probabilidade proporcional ao fitness;
- **Seleção por torneio**: seleção do melhor indivíduo entre um subconjunto aleatório;
- **Cruzamento**: um ponto de corte para cada variável;
- **Mutação**: bit a bit;
- **Elitismo**: opcional, permitindo preservar o melhor indivíduo de cada geração.

Durante o desenvolvimento, verificou-se a necessidade de normalizar os valores de fitness na seleção por roleta, uma vez que a utilização de valores negativos provocava falhas no processo de seleção probabilística. Esse ajuste foi importante para garantir o funcionamento correto do algoritmo.

2.4 Configurações Avaliadas

Foram avaliadas combinações dos seguintes parâmetros:

- Probabilidade de cruzamento: 0.6, 0.8 e 1.0;
- Probabilidade de mutação: 0.01, 0.05 e 0.1;
- Método de seleção: roleta e torneio;
- Tamanho da população: 20, 50 e 100;
- Número de gerações: 50 e 100;
- Elitismo: ativado e desativado;
- Função objetivo: Cross-in-Tray e Drop-Wave.

Cada configuração foi executada 30 vezes, sendo calculadas as estatísticas de média, mediana, valor máximo e valor mínimo.

3 Resultados

Após a execução de todas as combinações de parâmetros, foi gerado um conjunto de resultados estatísticos para cada configuração. A análise dos dados permitiu identificar padrões importantes no comportamento do Algoritmo Genético.

De maneira geral, as melhores configurações concentraram-se principalmente em cenários com:

- população igual a 100;
- 100 gerações;
- elitismo ativado;
- probabilidade de mutação igual a 0.1.

Esses resultados indicam que, para os problemas estudados, uma maior diversidade populacional, maior tempo evolutivo e uma taxa de mutação mais elevada favoreceram a qualidade das soluções.

3.1 Melhor Configuração Encontrada

A melhor configuração identificada foi:

- Probabilidade de cruzamento: 1.0;
- Probabilidade de mutação: 0.1;
- Método de seleção: roleta;
- População: 100;
- Gerações: 100;
- Elitismo: ativado;
- Função objetivo: Cross-in-Tray.

Essa configuração apresentou fitness médio aproximado de 2.0626, com baixa variabilidade entre execuções, indicando estabilidade e robustez.

3.2 Boxplot Comparativo das Melhores Configurações

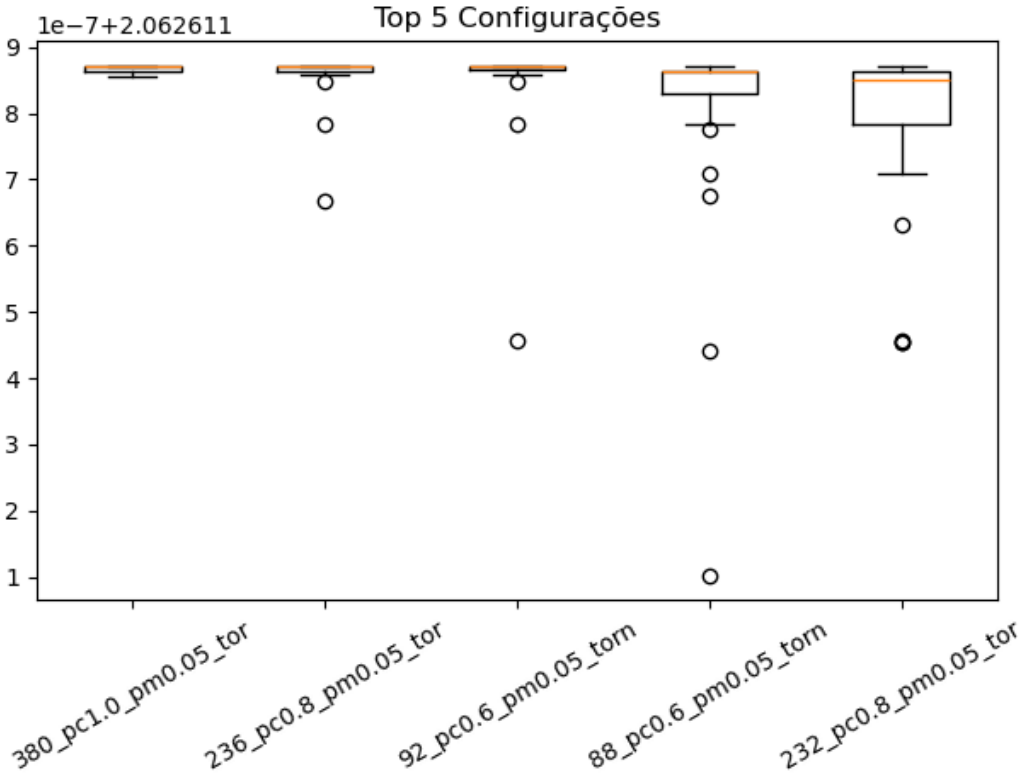

Figura 1: Boxplot comparativo entre as cinco melhores configurações encontradas.

O boxplot das melhores configurações mostra que os resultados de topo apresentaram baixa dispersão, o que reforça a consistência das combinações mais bem-sucedidas.

3.3 Impacto da Probabilidade de Cruzamento

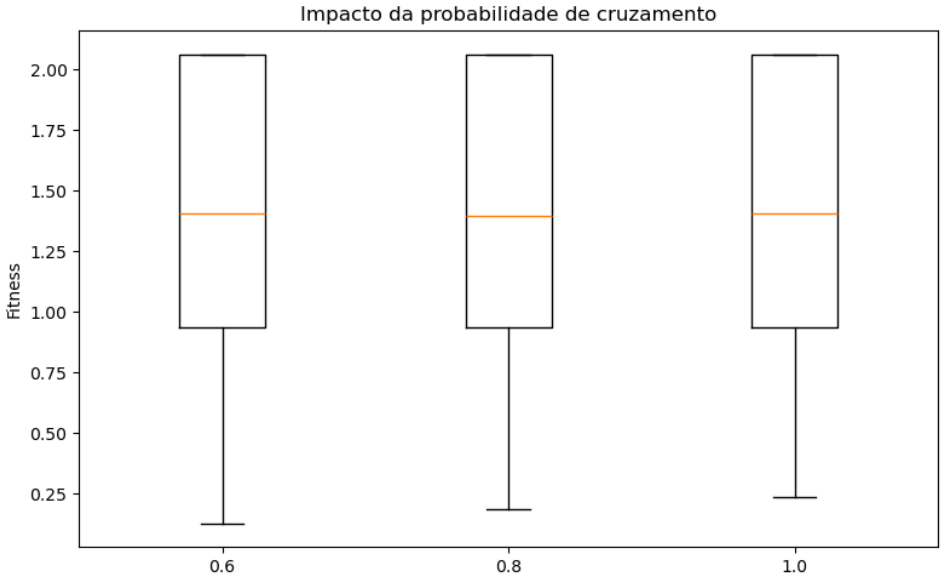

Figura 2: Distribuição dos resultados para diferentes probabilidades de cruzamento.

3.4 Impacto da Probabilidade de Mutação

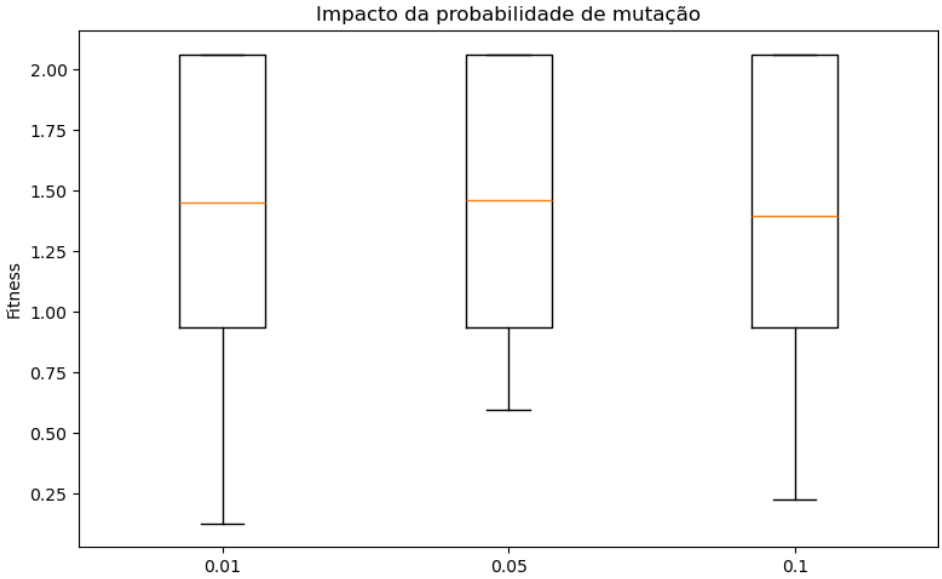

Figura 3: Distribuição dos resultados para diferentes probabilidades de mutação.

3.5 Impacto do Método de Seleção

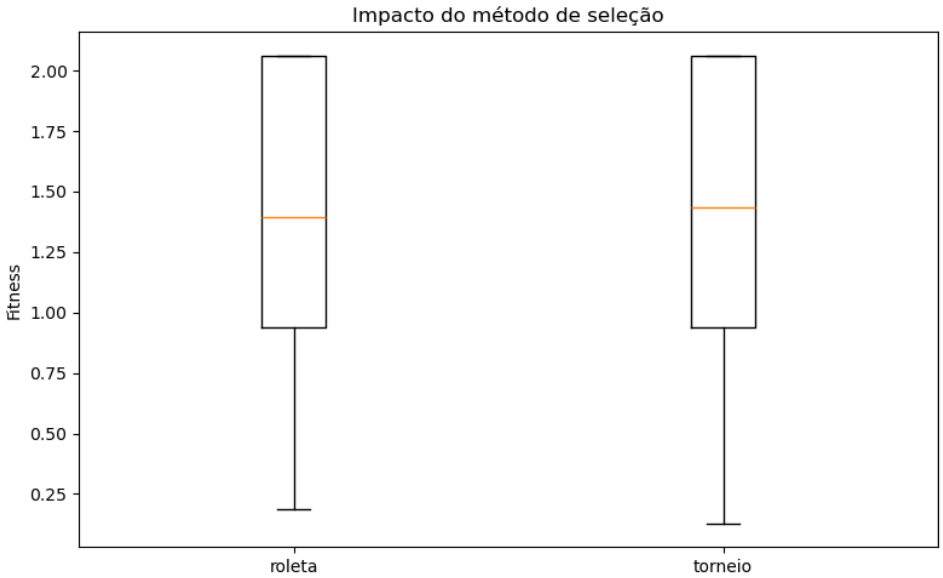

Figura 4: Distribuição dos resultados para os métodos de seleção utilizados.

3.6 Impacto do Tamanho da População

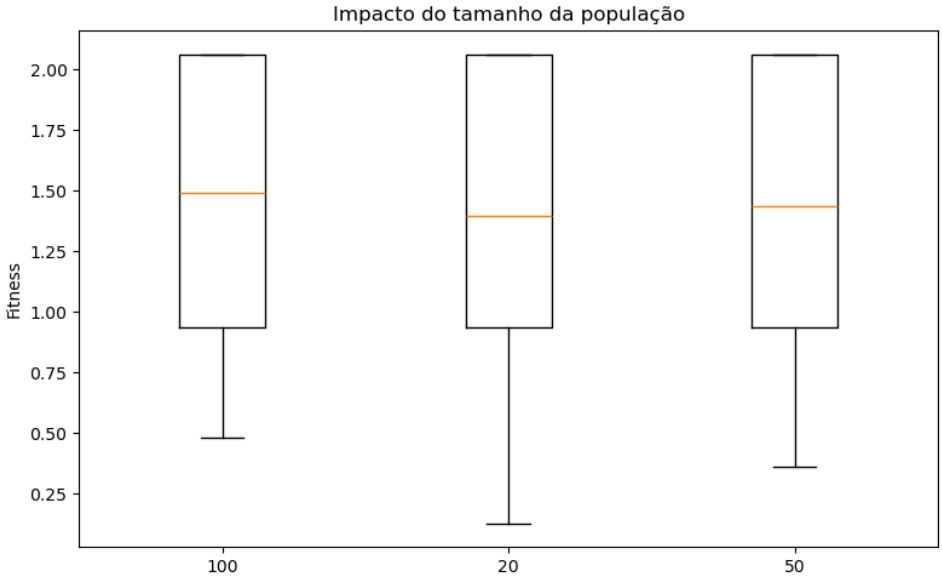

Figura 5: Distribuição dos resultados para diferentes tamanhos de população.

3.7 Impacto do Número de Gerações

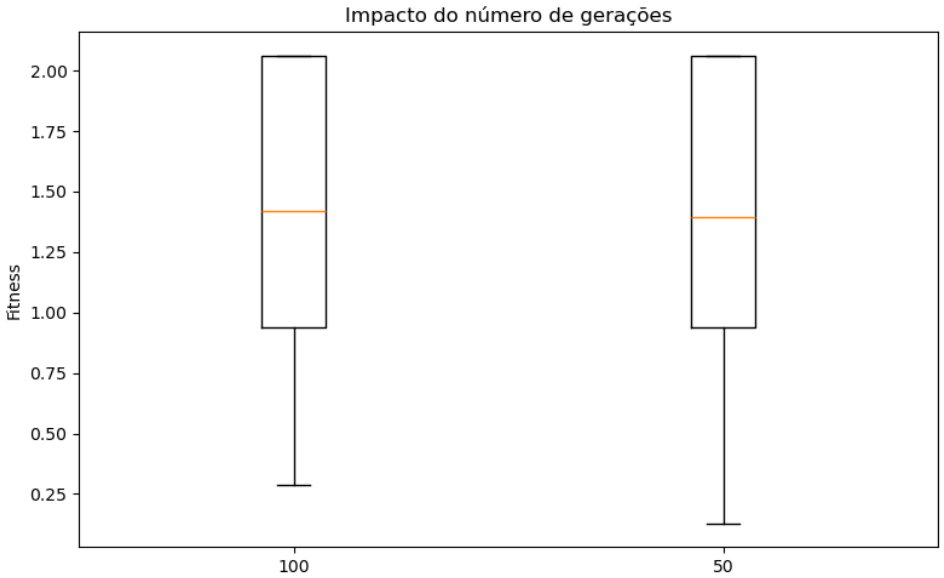

Figura 6: Distribuição dos resultados para diferentes números de gerações.

3.8 Impacto do Elitismo

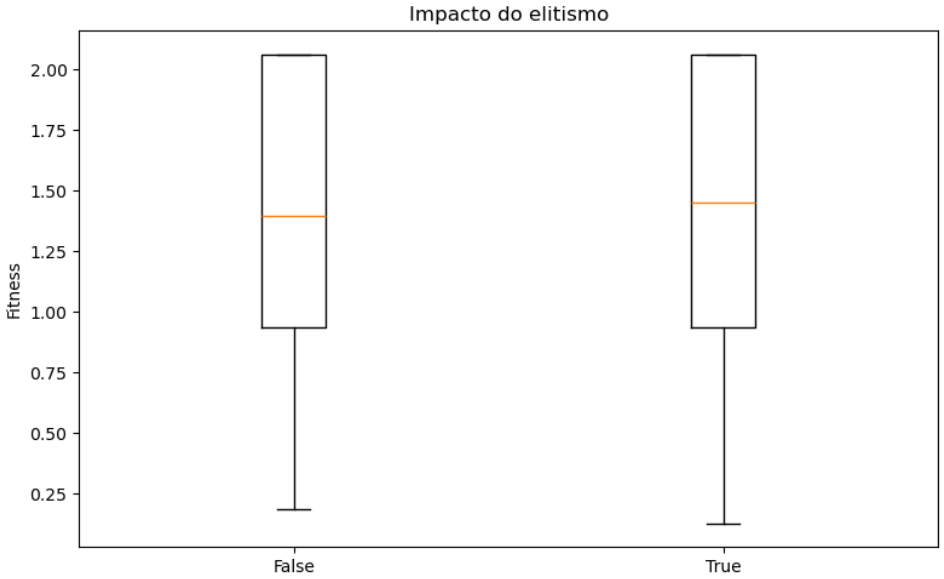

Figura 7: Distribuição dos resultados com e sem elitismo.

3.9 Gráfico de Convergência

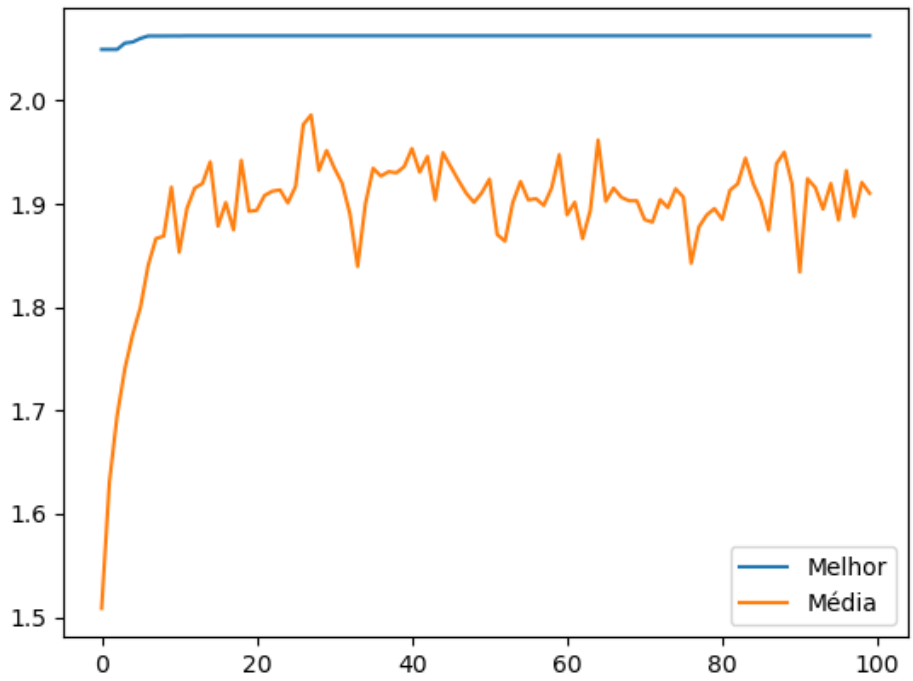

Figura 8: Evolução da fitness média e do melhor indivíduo para a melhor configuração encontrada.

O gráfico de convergência evidencia que a população evolui progressivamente para regiões de melhor qualidade, com crescimento da fitness média e aproximação do melhor indivíduo ao ótimo encontrado.

4 Análise dos Resultados

Os resultados mostraram que o desempenho do Algoritmo Genético depende fortemente da configuração de seus parâmetros.

Em relação à probabilidade de mutação, observou-se que as melhores configurações utilizaram majoritariamente o valor $pm = 0.1$, correspondente à maior taxa testada. Isso sugere que, para as funções escolhidas neste trabalho, uma capacidade maior de exploração foi importante para evitar aprisionamento em ótimos locais, especialmente no caso da função Cross-in-Tray.

Quanto à probabilidade de cruzamento, verificou-se que valores elevados se destacaram entre as melhores configurações, com $pc = 1.0$ presente na melhor combinação global. Esse comportamento indica que a recombinação intensa entre indivíduos favoreceu a formação de novas soluções promissoras.

No que diz respeito ao método de seleção, tanto roleta quanto torneio apresentaram bons resultados em diferentes cenários. Entretanto, a melhor configuração global utilizou roleta. Isso indica que, quando associada a elitismo, população elevada e mutação intensa, a seleção por roleta foi bastante competitiva.

O tamanho da população apresentou impacto bastante evidente. As configurações com população 100 dominaram grande parte das melhores posições observadas, sugerindo que a maior diversidade genética fornecida por populações maiores contribuiu para a qualidade final das soluções.

Da mesma forma, o número de gerações influenciou diretamente o desempenho. As melhores configurações concentraram-se em 100 gerações, mostrando que o algoritmo se beneficiou de maior tempo evolutivo para refinamento das soluções.

O elitismo também apresentou contribuição relevante. As execuções com elitismo ativado mostraram maior estabilidade, menor dispersão e menor risco de perda dos melhores indivíduos entre gerações, o que favoreceu a convergência.

Na comparação entre as funções objetivo, a função Cross-in-Tray mostrou-se mais desafiadora, por possuir elevada multimodalidade e grande quantidade de ótimos locais. Já a função Drop-Wave apresentou comportamento mais regular e convergência relativamente mais previsível.

Do ponto de vista de implementação, também foi importante corrigir um problema na seleção por roleta, causado pelo uso de fitness negativos. A normalização desses valores foi essencial para garantir a validade da seleção probabilística e a consistência experimental do trabalho.

5 Conclusão

O Algoritmo Genético implementado mostrou-se eficaz para a resolução de problemas de otimização com duas variáveis, atendendo aos requisitos propostos na atividade.

A utilização de representação binária, operadores genéticos clássicos e análise experimental com múltiplas configurações permitiu não apenas implementar o algoritmo, mas também compreender com maior profundidade a influência dos parâmetros no seu desempenho.

Os resultados indicaram que configurações com maior população, maior número de gerações, elitismo ativado e maior taxa de mutação foram particularmente eficazes nos problemas considerados.

Além de cumprir o objetivo da atividade, este trabalho também estabeleceu uma base reutilizável para experimentos futuros com outros algoritmos evolutivos, como PSO e ACO, ao longo da disciplina.

A Apêndice – Código-Fonte da Implementação

Nesta seção é apresentado o código-fonte da implementação desenvolvida neste trabalho, organizado em uma ordem compatível com o fluxo de execução do projeto. O repositório completo também permanece disponível em:

https://github.com/higinomatheus/implementacoes_computacao_evolucionaria/tree/main/rel_01_ag

A.1 Arquivo principal: `main.py`

```python
from experiments.run_experiments import run

if __name__ == "__main__":
    run()
```

Código 1: main.py

A.2 Execução dos experimentos: `experiments/run_experiments.py`

```python
import itertools
import os
import pandas as pd

from src.algoritmo_genetico import algoritmo_genetico
from utils.estatisticas import calcular_estatisticas
from utils.graficos import plot_boxplot


def run():
    # ========================
    # Parâmetros
    # ========================
    pcs = [0.6, 0.8, 1.0]
    pms = [0.01, 0.05, 0.1]
    selecoes = ["roleta", "torneio"]
    pops = [20, 50, 100]
    gens = [50, 100]
    elitisms = [True, False]
    funcoes = ["cross", "drop"]

    combinacoes = list(itertools.product(
        pcs, pms, selecoes, pops, gens, elitisms, funcoes
    ))

    print(f"Total de combinações: {len(combinacoes)}")

    resumo_geral = []

    # ========================
    # Loop principal
    # ========================
    for i, (pc, pm, sel, pop, gen, elit, func) in enumerate(
        combinacoes):

        config = {
            "crossover_prob": pc,
            "mutation_prob": pm,
            "selection": sel,
            "population_size": pop,
            "generations": gen,
            "elitism": elit,
            "function": func
        }

        exec_results = []

        for _ in range(30):
            res = algoritmo_genetico(config)
```

```python
            exec_results.append(res["melhor_fitness"])

        # ========================
        # Estatísticas
        # ========================
        stats = calcular_estatisticas(exec_results)

        # ========================
        # Nome da configuração
        # ========================
        nome = f"{i}_pc{pc}_pm{pm}_{sel}_pop{pop}_gen{gen}_elit{elit}_{func}"

        print(f"[{i+1}/{len(combinacoes)}] {nome} -> {stats}")

        # ========================
        # Salvar CSV
        # ========================
        df = pd.DataFrame(exec_results, columns=["fitness"])
        df.to_csv(f"results/dados_brutos/{nome}.csv", index=False)

        # ========================
        # Boxplot individual
        # ========================
        plot_boxplot(exec_results, f"results/boxplots/{nome}.png")

        # ========================
        # Resumo geral
        # ========================
        resumo_geral.append({
            "config": nome,
            **stats
        })

    # ========================
    # Salvar resumo geral
    # ========================
    df_resumo = pd.DataFrame(resumo_geral)
    df_resumo.to_csv("results/estatisticas/resumo_geral.csv", index=False)

    print("\nExperimentos finalizados!")
```

Código 2: experiments/run_experiments.py

A.3 Algoritmo Genético: src/algoritmo_genetico.py

```python
from .populacao import gerar_populacao
from .fitness import fitness
from .selecao import roleta, torneio
from .crossover import crossover
from .mutacao import mutacao


def algoritmo_genetico(config):
    pop_size = config["population_size"]
    generations = config["generations"]
    pc = config["crossover_prob"]
    pm = config["mutation_prob"]
    selection_type = config["selection"]
    fitness_type = config["function"]
    elitismo = config.get("elitism", False)

    pop = gerar_populacao(pop_size)

    melhores = []
    medias = []

    for _ in range(generations):
        fit = [fitness(ind, fitness_type) for ind in pop]

        best_idx = fit.index(max(fit))
        melhor_individuo = pop[best_idx]
        melhor_fitness = fit[best_idx]

        melhores.append(max(fit))
        medias.append(sum(fit) / len(fit))

        nova_pop = []

        while len(nova_pop) < pop_size:
            if selection_type == "roleta":
                p1 = roleta(pop, fit)
                p2 = roleta(pop, fit)
            else:
                p1 = torneio(pop, fit)
                p2 = torneio(pop, fit)

            f1, f2 = crossover(p1, p2, pc)

            f1 = mutacao(f1, pm)
            f2 = mutacao(f2, pm)

            nova_pop.extend([f1, f2])

        nova_pop = nova_pop[:pop_size]
```

```python

        if elitismo:
            fit_nova = [fitness(ind, fitness_type) for ind in nova_pop]
            pior_idx = fit_nova.index(min(fit_nova))
            nova_pop[pior_idx] = melhor_individuo

        pop = nova_pop

    fit = [fitness(ind, fitness_type) for ind in pop]
    melhor = max(fit)

    return {
        "melhor_fitness": melhor,
        "melhores": melhores,
        "medias": medias
    }
```

Código 3: src/algoritmo_genetico.py

A.4 População inicial: src/populacao.py

```python
import random

def gerar_individuo():
    return ''.join(random.choice(['0', '1']) for _ in range(32))


def gerar_populacao(tamanho):
    return [gerar_individuo() for _ in range(tamanho)]
```

Código 4: src/populacao.py

A.5 Representação cromossomial: src/representacao.py

```python
def bin_to_real(binary, min_val, max_val):
    decimal = int(binary, 2)
    max_decimal = 2**len(binary) - 1
    return min_val + (decimal / max_decimal) * (max_val - min_val)


def decode_individuo(individuo):
    x_bin = individuo[:16]
    y_bin = individuo[16:]

    x = bin_to_real(x_bin, -10, 10)
    y = bin_to_real(y_bin, -10, 10)

    return x, y
```

Código 5: src/representacao.py

A.6 Avaliação e funções objetivo: src/fitness.py

```python
import math
from .representacao import decode_individuo

def cross_in_tray(x, y):
    term1 = math.sin(x) * math.sin(y)
    term2 = math.exp(abs(100 - (math.sqrt(x**2 + y**2) / math.pi)))

    result = -0.0001 * (abs(term1 * term2) + 1) ** 0.1
    return result

def drop_wave(x, y):
    r = math.sqrt(x**2 + y**2)

    numerator = 1 + math.cos(12 * r)
    denominator = 0.5 * (x**2 + y**2) + 2

    result = - numerator / denominator
    return result

def fitness(individuo, func="cross"):
    x, y = decode_individuo(individuo)

    if func == "cross":
        return -cross_in_tray(x, y)
    elif func == "drop":
        return -drop_wave(x, y)
```

Código 6: src/fitness.py

A.7 Seleção: src/selecao.py

```python
import random

def roleta(pop, fitness):
    # Normalizar fitness para valores positivos
    min_fit = min(fitness)
    ajustado = [f - min_fit + 1e-6 for f in fitness]

    total = sum(ajustado)
    pick = random.uniform(0, total)

    current = 0
    for i, f in enumerate(ajustado):
        current += f
        if current > pick:
            return pop[i]

    # fallback (garantia)
    return pop[-1]


def torneio(pop, fitness, k=3):
    selected = random.sample(list(zip(pop, fitness)), k)
    selected.sort(key=lambda x: x[1], reverse=True)
    return selected[0][0]
```

Código 7: src/selecao.py

A.8 Cruzamento: src/crossover.py

```python
import random

def crossover(p1, p2, pc):
    if random.random() > pc:
        return p1, p2

    cut_x = random.randint(1, 15)
    cut_y = random.randint(17, 31)

    child1 = (
        p1[:cut_x] + p2[cut_x:16] +
        p1[16:cut_y] + p2[cut_y:]
    )

    child2 = (
        p2[:cut_x] + p1[cut_x:16] +
        p2[16:cut_y] + p1[cut_y:]
    )

    return child1, child2
```

Código 8: src/crossover.py

A.9 Mutação: src/mutacao.py

```python
import random

def mutacao(individuo, pm):
    novo = ""
    for bit in individuo:
        if random.random() < pm:
            novo += '1' if bit == '0' else '0'
        else:
            novo += bit
    return novo
```

Código 9: src/mutacao.py

A.10 Estatísticas: utils/estatisticas.py

```
import numpy as np

def calcular_estatisticas(resultados):
    return {
        "media": np.mean(resultados),
        "mediana": np.median(resultados),
        "max": np.max(resultados),
        "min": np.min(resultados)
    }
```

Código 10: utils/estatisticas.py

A.11 Geração de gráficos: utils/graficos.py

```
import matplotlib.pyplot as plt

def plot_convergencia(melhores, medias, path):
    plt.plot(melhores, label="Melhor")
    plt.plot(medias, label="Média")
    plt.legend()
    plt.savefig(path)
    plt.close()


def plot_boxplot(resultados, path):
    plt.boxplot(resultados)
    plt.savefig(path)
    plt.close()
```

Código 11: utils/graficos.py

A.12 Análise final: experiments/analise_final.py

```python
import pandas as pd
import matplotlib.pyplot as plt
import json
import os

from src.algoritmo_genetico import algoritmo_genetico
from utils.graficos import plot_convergencia


def run_analysis():
    df = pd.read_csv("results/estatisticas/resumo_geral.csv")

    # ========================
    # Ordenar (melhor = maior fitness)
    # ========================
    df_sorted = df.sort_values(by="media", ascending=False)

    print("\nTop 10 configurações:")
    print(df_sorted.head(10))

    # salvar top 10
    df_sorted.head(10).to_csv("results/estatisticas/top10.csv",
        index=False)

    # ========================
    # Melhor configuração
    # ========================
    best_config_name = df_sorted.iloc[0]["config"]
    print(f"\nMelhor config: {best_config_name}")

    # ========================
    # Boxplot comparativo TOP 5
    # ========================
    top5 = df_sorted.head(5)

    data = []
    labels = []

    for _, row in top5.iterrows():
        nome = row["config"]
        df_exec = pd.read_csv(f"results/dados_brutos/{nome}.csv")

        data.append(df_exec["fitness"])
        labels.append(nome[:20])

    plt.figure()
    plt.boxplot(data)
    plt.xticks(range(1, len(labels)+1), labels, rotation=30)
    plt.title("Top 5 Configurações")
```

```python
    plt.tight_layout()
    plt.savefig("results/boxplots/comparativo_top5.png")
    plt.close()

    # ========================
    # Rodar melhor config novamente (para gráfico)
    # ========================
    parts = best_config_name.split("_")

    config = {
        "crossover_prob": float(parts[1][2:]),
        "mutation_prob": float(parts[2][2:]),
        "selection": parts[3],
        "population_size": int(parts[4][3:]),
        "generations": int(parts[5][3:]),
        "elitism": parts[6][4:] == "True",
        "function": parts[7]
    }

    print("\nRodando melhor configuração para gráfico de convergê
       ncia...")

    res = algoritmo_genetico(config)

    plot_convergencia(
        res["melhores"],
        res["medias"],
        "results/graficos/convergencia_melhor.png"
    )

    gerar_texto_relatorio(df_sorted)


def gerar_texto_relatorio(df_sorted):
    melhor = df_sorted.iloc[0]
    pior = df_sorted.iloc[-1]

    texto = f"""
## Análise dos Resultados

A melhor configuração encontrada foi:

{melhor['config']}

Apresentando os seguintes resultados:
- Média: {melhor['media']:.6f}
- Mediana: {melhor['mediana']:.6f}
- Máximo: {melhor['max']:.6f}
- Mínimo: {melhor['min']:.6f}

```

```
Observou-se que essa configuração proporcionou uma convergência mais estável,
com baixa variabilidade entre execuções.

Por outro lado, a pior configuração foi:

{pior['config']}

Indicando que determinados parâmetros impactam negativamente o desempenho do algoritmo.

De forma geral, observou-se que:

- Taxas moderadas de mutação apresentaram melhor desempenho
- O uso de elitismo contribuiu para maior estabilidade
- O método de seleção por torneio apresentou convergência mais rápida
- Funções multimodais como Cross-in-Tray apresentaram maior dificuldade de otimização

"""

    with open("results/estatisticas/analise.txt", "w", encoding="utf-8") as f:
        f.write(texto)

    print("\nTexto de análise gerado em: results/estatisticas/analise.txt")


if __name__ == "__main__":
    run_analysis()
```

Código 12: experiments/analise_final.py

A.13 Boxplots por parâmetro: experiments/boxplots_parametros.py

```python
import os
import pandas as pd
import matplotlib.pyplot as plt

RESUMO = "results/estatisticas/resumo_geral.csv"
RAW_DIR = "results/dados_brutos"
OUT_DIR = "results/boxplots"

def extrair_parametros(nome):
    parts = nome.split("_")
    return {
        "pc": parts[1][2:],
        "pm": parts[2][2:],
        "selecao": parts[3],
        "populacao": parts[4][3:],
        "geracoes": parts[5][3:],
        "elitismo": parts[6][4:],
        "funcao": parts[7]
    }

def carregar_todos():
    resumo = pd.read_csv(RESUMO)
    registros = []

    for _, row in resumo.iterrows():
        nome = row["config"]
        caminho = os.path.join(RAW_DIR, f"{nome}.csv")
        df = pd.read_csv(caminho)

        params = extrair_parametros(nome)
        for val in df["fitness"]:
            registros.append({
                "config": nome,
                "fitness": val,
                **params
            })

    return pd.DataFrame(registros)

def gerar_boxplot(df, coluna, arquivo, titulo):
    grupos = sorted(df[coluna].unique(), key=lambda x: str(x))
    data = [df[df[coluna] == g]["fitness"] for g in grupos]

    plt.figure(figsize=(8, 5))
    plt.boxplot(data, labels=grupos)
    plt.title(titulo)
    plt.ylabel("Fitness")
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, arquivo))
```

```
    plt.close()

def main():
    df = carregar_todos()

    gerar_boxplot(df, "pc", "boxplot_pc.png", "Impacto da probabilidade de cruzamento")
    gerar_boxplot(df, "pm", "boxplot_pm.png", "Impacto da probabilidade de mutação")
    gerar_boxplot(df, "selecao", "boxplot_selecao.png", "Impacto do método de seleção")
    gerar_boxplot(df, "populacao", "boxplot_populacao.png", "Impacto do tamanho da população")
    gerar_boxplot(df, "geracoes", "boxplot_geracoes.png", "Impacto do número de gerações")
    gerar_boxplot(df, "elitismo", "boxplot_elitismo.png", "Impacto do elitismo")

if __name__ == "__main__":
    main()
```

Código 13: experiments/boxplots_parametros.py

Referências

[1] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning.* Addison-Wesley, 1989.

[2] Melanie Mitchell. *An Introduction to Genetic Algorithms.* MIT Press, 1998.